Deblur-NeRF Acceleration Project

Project Overview

This project focuses on accelerating Deblur-NeRF training while maintaining high reconstruction quality. We experiment with multiple optimization techniques such as sampling reduction, hash encoding, and kernel pruning, and analyze their impact on both runtime and visual fidelity.

Dataset

We use datasets from the official Deblur-NeRF repository:
https://limacv.github.io/deblurnerf/

Each dataset contains:

- Blurred images
- Camera poses (`poses_bounds.npy`)

Example used in this project: `blurball`

Project Structure

```
Deblur-NeRF-master/
│
├── configs/
├── data/
│   └── blurball/
├── logs_exp/
│   └── <experiment_name>/
├── run_nerf.py
```

Outputs

After training/rendering, results are saved under:

```
logs_exp/<experiment_name>/
```

This includes:

- Checkpoints (`.tar` files)
- Rendered images (`testset_XXXXX`)
- Rendered videos (`renderonly_path_XXXXX`)

Setup

Run Path

Run from:

```
project_322879651_212356331/Deblur-NeRF-master
```

Install Dependencies

```
pip install torch torchvision --upgrade
pip install -r requirements.txt
```

Install tiny-cuda-nn

```
pip install git+https://github.com/NVlabs/tiny-cuda-
nn/#subdirectory=bindings/torch --no-build-isolation
```

Configuration

> Note: All examples use the `blurball` dataset.

To update a config file for a new dataset:

1. Update `num_input_views` according to the number of images.
2. Change `expname` as desired.
3. Update `datadir` to point to the dataset directory. (It is recommended to copy dataset into `Deblur-NeRF-master/data/`)

Example config files can be found under:

```
configs/blurball/
configs/blurdecoration/
```

Config file format:

```
tx_***_full_crf.txt
```

Training

```
python run_nerf.py --config configs/blurball/tx_blurball_full_crf.txt
```

Rendering

Render Video

```
python run_nerf.py --config configs/blurball/tx_blurball_full_crf.txt --
render_only
```

Render Test Images

```
python run_nerf.py --config configs/blurball/tx_blurball_full_crf.txt --
render_only --render_test
```

Checkpoint Usage

To render from a specific training checkpoint:

```
--ft_path <path_to_checkpoint>
```

Example:

```
--ft_path ./logs_exp/blurball_tcnn_crf_v5/030000.tar
```

Full command:

```
python run_nerf.py \
  --config configs/blurball/tx_blurball_full_crf.txt \
  --render_only \
  --ft_path ./logs_exp/blurball_tcnn_crf_v5/030000.tar
```

Results are saved under:

```
Deblur-NeRF-master/logs_exp/
```

Running on a Different Dataset

1. Copy dataset from:

```
deblurnerf_dataset/real_camera_motion_blur
```

to:

```
Deblur-NeRF-master/data/
```

2. Update config file:

```
datadir = ./data/<dataset_name>
```

3. Run training as usual.

Known Issues

- Dataset must be in LLFF format (requires `poses_bounds.npy`)
- Some acceleration methods (e.g., occupancy grid, hash encoding) may degrade quality

---

Results Summary

- Achieved ~4× speedup (**1.7s → 0.42s per iteration**)
- Most convergence occurs by ~100K iterations, with minimal improvement beyond that point